

Copyright 2018 The TensorFlow Authors.

In [1]:

```
##@title Licensed under the Apache License, Version 2.0 (the "License");
## you may not use this file except in compliance with the License.
## You may obtain a copy of the License at
##
## https://www.apache.org/licenses/LICENSE-2.0
##
## Unless required by applicable law or agreed to in writing, software
## distributed under the License is distributed on an "AS IS" BASIS,
## WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
## See the License for the specific language governing permissions and
## limitations under the License.
```

In [99]:

```
# pip install tensorflow==2.12.3.*
```

Looking in indexes: <https://pypi.org/simple>, (<https://pypi.org/simple>), <https://pip.repos.neuron.amazonaws.com> (<https://pip.repos.neuron.amazonaws.com>)

ERROR: Could not find a version that satisfies the requirement tensorflow==2.12.3.* (from versions: 1.15.5, 2.8.0rc0, 2.8.0rc1, 2.8.0, 2.8.1, 2.8.2, 2.8.3, 2.8.4, 2.9.0rc0, 2.9.0rc1, 2.9.0rc2, 2.9.0, 2.9.1, 2.9.2, 2.9.3, 2.10.0rc0, 2.10.0rc1, 2.10.0rc2, 2.10.0rc3, 2.10.0, 2.10.1, 2.11.0rc0, 2.11.0rc1, 2.11.0rc2, 2.11.0, 2.11.1, 2.12.0rc0, 2.12.0rc1, 2.12.0, 2.12.1, 2.13.0rc0, 2.13.0rc1, 2.13.0rc2, 2.13.0)

ERROR: No matching distribution found for tensorflow==2.12.3.*

Note: you may need to restart the kernel to use updated packages.

In [3]:

```
# import shutil
# os.remove("./train.zip")
# shutil.rmtree("./train")
```

Image classification

In [4]:

```
# pip list
```



[View on TensorFlow.org](https://www.tensorflow.org/tutorials/images/classification)

(<https://www.tensorflow.org/tutorials/images/classification>) (<https://colab.research.google.com/github/tensorflow/docs/blob/master>)

This tutorial shows how to classify images of flowers using a `tf.keras.Sequential` model and load data using `tf.keras.utils.image_dataset_from_directory`. It demonstrates the following concepts:

- Efficiently loading a dataset off disk.
- Identifying overfitting and applying techniques to mitigate it, including data augmentation and dropout.

This tutorial follows a basic machine learning workflow:

1. Examine and understand data
2. Build an input pipeline
3. Build the model
4. Train the model
5. Test the model
6. Improve the model and repeat the process

In addition, the notebook demonstrates how to convert a [saved model \(../..../guide/saved_model.ipynb\)](#) to a [TensorFlow Lite \(https://www.tensorflow.org/lite/\)](https://www.tensorflow.org/lite/) model for on-device machine learning on mobile, embedded, and IoT devices.

Setup

Import TensorFlow and other necessary libraries:

In []:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import random
import PIL
import tensorflow as tf
import os
import boto3
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers.legacy import Adam
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

In [132]:

```
tf.version.VERSION
```

Out[132]:

```
'2.13.0'
```

In [133]:

```
# # import boto3
# # # import awscli as wr

# # # Define your AWS access key and secret key
# aws_access_key_id = "AKIASNSREPJV4LF3IK4V"
# aws_secret_access_key = "KyWYxxbcekLFibz59jwdxYaXByKZcu4sGWYqmQgF"

# # # Use boto3 to create a session with your AWS credentials
# session = boto3.Session(
#     aws_access_key_id=aws_access_key_id,
#     aws_secret_access_key=aws_secret_access_key
# )

# # # Use awscli to connect to your S3 bucket
# # bucket_name = "classifyraw"
# # subfolder = "MRI_T2"

# # # List the contents of your S3 bucket
# # s3 = session.resource("s3")
# # result = s3.Bucket(bucket_name)

# # for my_bucket_object in result.objects.filter(Prefix="modeling_mri_t2/train/"):
# #     print(my_bucket_object.key)

# # # Print the contents of the bucket
# # # if "Contents" in result:
# # #     for obj in result["Contents"]:
# # #         print(obj)
# # # else:
# # #     print("The bucket is empty.")

# import boto3
# from os import path, makedirs
# from botocore.exceptions import ClientError
# from boto3.exceptions import S3TransferFailedError
```

In [134]:

```
# import zipfile
# with zipfile.ZipFile("./train.zip", 'r') as zip_ref:
#     zip_ref.extractall("./")
```

Download and explore the dataset

In [135]:

```
import pathlib

data_dir = "/Users/SeanKelly1/Documents/FinalThesis(2023Ulster)/Raw/All/modeling_mr

data_dir = '/home/ec2-user/SageMaker/train'
data_dir = pathlib.Path(data_dir)
```

In [136]:

```
import os
print(len(os.listdir(data_dir)))
```

3

In [137]:

```
len(list(data_dir.glob('demented/*.jpg')))
```

Out[137]:

11502

After downloading, you should now have a copy of the dataset available. There are 3,670 total images:

In [138]:

```
image_count = len(list(data_dir.glob('*/*.jpg')))
print(image_count)
```

23003

Resize all the images

In [139]:

```
# from matplotlib import image as im
# import random
# all_images = list(data_dir.glob('*/*.jpg'))
# print(len(all_images))

# for each in all_images:
# #     name = str(each).split("/")[9]
# #     first = name.split(".")[0]
# #     second = name.split(".")[1]
#     each = pathlib.Path(str(each))
# #     print(each)
# #     ima = im.imread('/home/ec2-user/SageMaker/train/not_demented/OAS31081_02_0.0')
#     image = PIL.Image.open(each)
#     image = PIL.ImageOps.grayscale(image)
# #     image = image.convert("RGB")
#     image = image.resize((224, 224))
#     image.save(str(each))
#     print("done")

# random_list = random.sample(range(0, 11503), demented_size)
# for temp in random_list:
#     each = not_demented[temp]
#     name = str(each).split("/")[9]
#     first = name.split(".")[0]
#     second = name.split(".")[1]
#     rename = f"/Users/SeanKelly1/Documents/FinalThesis(2023Ulster)/Raw/All/PIB/train/not_demented/OAS31081_02_0.0_{first}_{second}.jpg"
#     image = PIL.Image.open(str(each))
#     image = image.convert("RGB")
#     image = image.resize((126, 126))
#     image.save(rename)
#     print("done not demented")
```

In [140]:

```
# Imports PIL module
from PIL import Image

# open method used to open different extension image file
im = Image.open(r"/home/ec2-user/SageMaker/train/not_demented/OAS30335_TSE_0.0_32.jpg")

# This method will show image in any image viewer
im.size
```

Out[140]:

(128, 128)

Here are some demented:

In [141]:

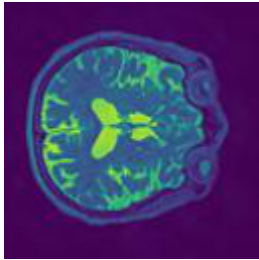
```
demented = list(data_dir.glob('demented/*'))  
img = PIL.Image.open(str(demented[0]))  
print(img.size)
```

(128, 128)

In [142]:

```
PIL.Image.open(str(demented[2]))
```

Out[142]:

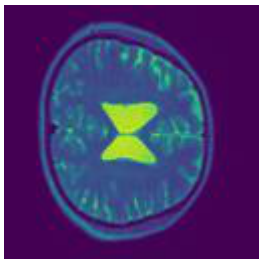


And some non_demented:

In [143]:

```
not_demented = list(data_dir.glob('not_demented/*'))  
PIL.Image.open(str(not_demented[0]))
```

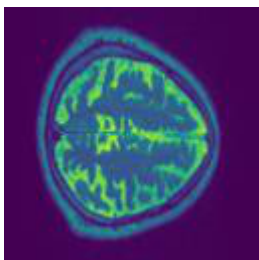
Out[143]:



In [144]:

```
PIL.Image.open(str(not_demented[1]))
```

Out[144]:



Create the test set from the full data

In [145]:

```
# # Create the test folder
# # From each class select random 10% of the total

# test_dem = "/Users/SeanKelly1/Documents/FinalThesis(2023Ulster)/Raw/All/modeling_n
# test_not_dem = "/Users/SeanKelly1/Documents/FinalThesis(2023Ulster)/Raw/All/modeli

# if os.path.exists(test_dem) or os.path.exists(test_not_dem):
#     print("above folders exists")
# else:
#     os.makedirs(test_dem)
#     os.makedirs(test_not_dem)
#     print("created")
```

In [146]:

```
print(data_dir)
```

```
/home/ec2-user/SageMaker/train
```

In [147]:

```
# import random

# random_list = []
# # Set a length of the list to 10

# # any random numbers from 0 to 1000
# random_list = random.sample(range(0, 12781), 1278)
# demented = list(data_dir.glob('demented/*'))
# not_demented = list(data_dir.glob('not_demented/*'))
# # print(len(demented))
# # print(len(not_demented))
# for each in random_list:
#     demented_test_filename = str(demented[each]).split("/")[9]
#     not_demented_test_filename = str(not_demented[each]).split("/")[9]
#     os.replace(demented[each], test_dem + "/" + demented_test_filename)
#     os.replace(not_demented[each], test_not_dem + "/" + not_demented_test_filename)

# print("done")
```

Load data using a Keras utility

Next, load these images off disk using the helpful `tf.keras.utils.image_dataset_from_directory` utility. This will take you from a directory of images on disk to a `tf.data.Dataset` in just a couple lines of code. If you like, you can also write your own data loading code from scratch by visiting the [Load and preprocess images \(./load_data/images.ipynb\)](#) tutorial.

Create a dataset

Define some parameters for the loader:

In [148]:

```
batch_size = 32
img_height = 128
img_width = 128
```

We split the data into train and validation in proportion of 4:1

In [149]:

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    color_mode = "rgb",
    label_mode = "int")
```

Found 23003 files belonging to 2 classes.
Using 18403 files for training.

In [150]:

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    color_mode = "rgb",
    label_mode = "int")
```

Found 23003 files belonging to 2 classes.
Using 4600 files for validation.

You can find the class names in the `class_names` attribute on these datasets. These correspond to the directory names in alphabetical order.

In [151]:

```
class_names = train_ds.class_names
print(class_names)
```

```
['demented', 'not_demented']
```

Visualize the data

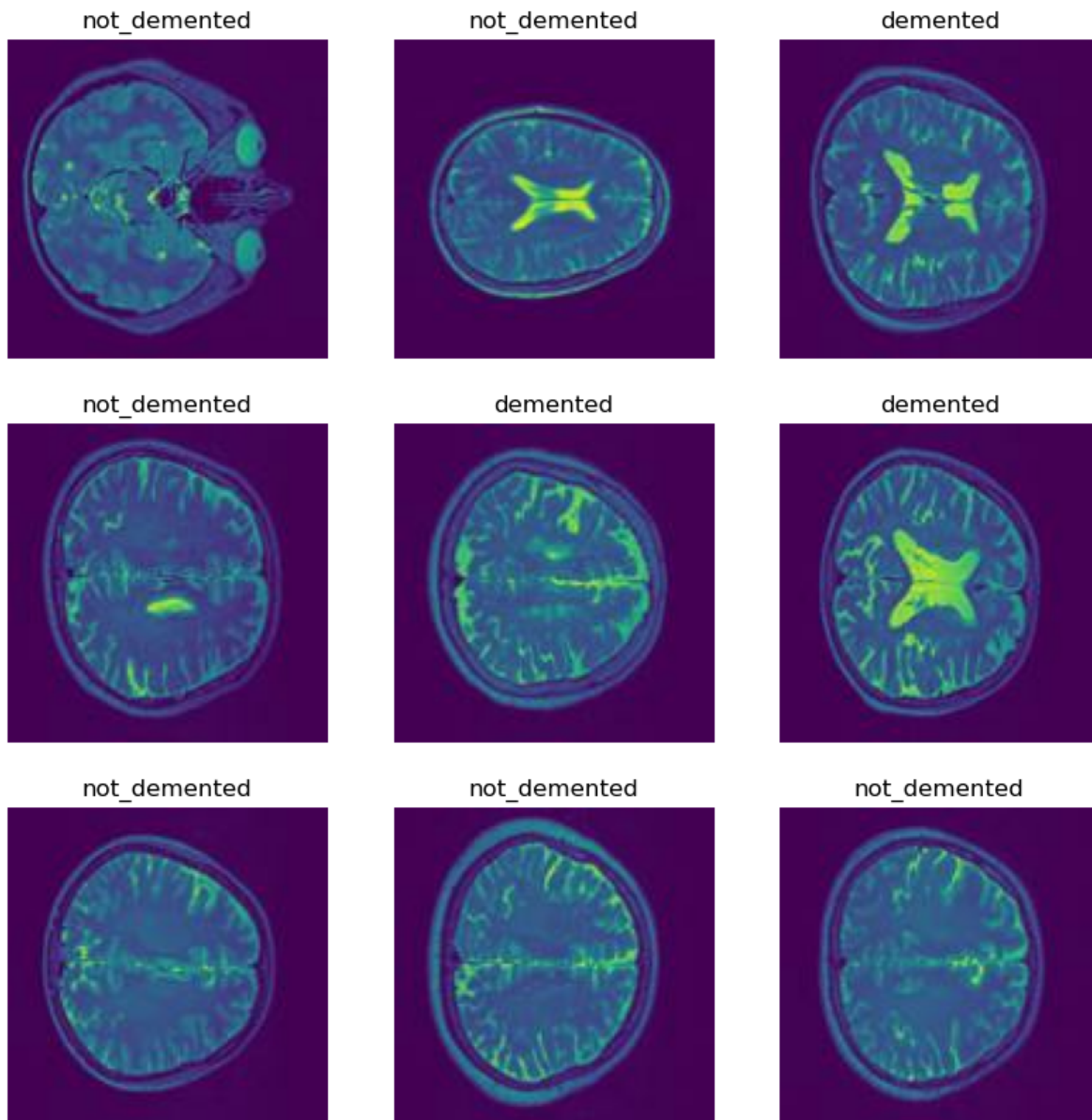
Here are the first nine images from the training dataset:

In [152]:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    print(labels)
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
```

```
tf.Tensor([1 1 0 1 0 0 1 1 1 0 1 0 1 1 1 0 0 1 1 1 0 0 0 1 1 0 0 1 0 1
0 1], shape=(32,), dtype=int32)
```



You will pass these datasets to the Keras `Model.fit` method for training later in this tutorial. If you like, you can also manually iterate over the dataset and retrieve batches of images:

In [153]:

```
for image_batch, labels_batch in train_ds:
    print(image_batch.shape)
    print(labels_batch.shape)
    break

(32, 128, 128, 3)
(32,)
```

The `image_batch` is a tensor of the shape `(32, 128, 128, 3)`. This is a batch of 32 images of shape `126x126x3` (the last dimension refers to color channels RGB). The `label_batch` is a tensor of the shape `(32,)`, these are corresponding labels to the 32 images.

You can call `.numpy()` on the `image_batch` and `labels_batch` tensors to convert them to a `numpy.ndarray`.

Configure the dataset for performance

Make sure to use buffered prefetching, so you can yield data from disk without having I/O become blocking. These are two important methods you should use when loading data:

- `Dataset.cache` keeps the images in memory after they're loaded off disk during the first epoch. This will ensure the dataset does not become a bottleneck while training your model. If your dataset is too large to fit into memory, you can also use this method to create a performant on-disk cache.
- `Dataset.prefetch` overlaps data preprocessing and model execution while training.

Interested readers can learn more about both methods, as well as how to cache data to disk in the *Prefetching* section of the [Better performance with the tf.data API \(../guide/data_performance.ipynb\)](#) guide.

In [154]:

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

Standardize the data

The RGB channel values are in the `[0, 255]` range. This is not ideal for a neural network; in general you should seek to make your input values small.

Here, you will standardize values to be in the `[0, 1]` range by using `tf.keras.layers.Rescaling`:

In [155]:

```
normalization_layer = layers.Rescaling(1./255)
```

There are two ways to use this layer. You can apply it to the dataset by calling `Dataset.map`:

In [156]:

```
normalized_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
image_batch, labels_batch = next(iter(normalized_ds))
first_image = image_batch[0]
# Notice the pixel values are now in `[0,1]`.
print(np.min(first_image), np.max(first_image))
```

0.0 0.95294124

Or, you can include the layer inside your model definition, which can simplify deployment. Use the second approach here.

Note: You previously resized images using the `image_size` argument of `tf.keras.utils.image_dataset_from_directory`. If you want to include the resizing logic in your model as well, you can use the `tf.keras.layers.Resizing` layer.

RANDOM FOREST MODEL

Create and Train the model

In [157]:

```
from sklearn.ensemble import RandomForestClassifier

# Generate Test data
import pathlib

train_dir = "/home/ec2-user/SageMaker/train"

print(os.listdir(train_dir))

demented = "/home/ec2-user/SageMaker/train/demented"
not_demented = "/home/ec2-user/SageMaker/train/not_demented"

print(len(os.listdir(demented)))

demented_data = os.listdir(demented)
not_demented_data = os.listdir(not_demented)

train_data = []
train_labels = []
for i in range(len(os.listdir(demented))):
    try:
        temp_file_d = demented_data[i]
        path = demented + "/" + temp_file_d
        temp = pathlib.Path(path)
        image_dem = tf.keras.utils.load_img(temp)
        train_data.append(image_dem)
        train_labels.append("demented")
    #     print(image_dem)
    #     print(list(zip(data, labels)))

    temp_file_nd = not_demented_data[i]
    path_temp = not_demented + "/" + temp_file_nd
    tempn = pathlib.Path(path_temp)
    image_ndem = tf.keras.utils.load_img(tempn)
    train_data.append(image_ndem)
    train_labels.append("not_demented")
except Exception as err :
    print(i, err)
    continue
print(len(train_data), len(train_labels))

['not_demented', '.DS_Store', 'demented']
11503
5155 cannot identify image file <_io.BytesIO object at 0x7fb1dbd07f60>
5166 cannot identify image file <_io.BytesIO object at 0x7fb1dcb12e80>
11502 list index out of range
23002 23002
```

In [158]:

```
print(train_data[0], train_labels[0])
print(train_data[1], train_labels[1])
```

```
<PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=128x128 at 0x7FB7B61E51E0> demented
<PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=128x128 at 0x7FB1DD843670> not_demented
```

In [159]:

```
# Shuffle the two lists
data_df = list(zip(train_data, train_labels))
random.shuffle(data_df)
print(len(data_df))
train_data, train_labels = zip(*data_df)
print(len(train_data), len(train_labels))
```

```
23002
23002 23002
```

In [160]:

```
nkan = [tf.expand_dims(x, 0) for x in train_data]
print(len(nkan), type(nkan))
useful = nkan.copy()
```

```
data_sh = list(zip(nkan, train_labels))
random.shuffle(data_sh)
print(len(data_sh))
train_data, train_labels = zip(*data_sh)
print(len(train_data), len(train_labels))
```

```
train_d = np.concatenate((train_data), axis=0)
print(train_d.shape)
```

```
23002 <class 'list'>
23002
23002 23002
(23002, 128, 128, 3)
```

In [161]:

```
print(np.min(train_d), np.max(train_d))
train_d = train_d/255
print(np.min(train_d), np.max(train_d))
```

```
0 255
0.0 1.0
```

In [162]:

```

from sklearn.ensemble import RandomForestClassifier

fir, sec, third, fourth = train_d.shape
print(sec, third, fourth)
train_d_r = train_d.reshape(fir, (sec*third*fourth))

print(train_d_r.shape)

# Create an instance of the RandomForestClassifier class:

model=RandomForestClassifier()

# Finally, let us proceed to train the model:

model.fit(train_d_r, train_labels)

```

```

128 128 3
(23002, 49152)

```

Out[162]:

```

▼ RandomForestClassifier
RandomForestClassifier()

```

In [163]:

```

dataset = tf.data.Dataset.from_tensor_slices(list(zip(train_d_r, train_labels))).bat

# for x, y in dataset:
#     print(x.shape, y)

```

```

-----
-----
TypeError                                Traceback (most recent call
last)
File ~/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/tensorflow/python/data/util/structure.py:104, in normalize_element(element, element_signature)
    103     if spec is None:
--> 104         spec = type_spec_from_value(t, use_fallback=False)
    105 except TypeError:
    106     # TypeError indicates it was not possible to compute a `TypeSpec` for
    107     # the value. As a fallback try converting the value to a tensor.

File ~/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/tensorflow/python/data/util/structure.py:507, in type_spec_from_value(element, use_fallback)
    504     logging.vlog(
--

```

In []:

```
train_to_list = train_d_r.tolist()  
print(train_)
```

In [165]:

```
import tensorflow_decision_forests as tfdf

# tf_dataset = tfdf.keras.pd_dataframe_to_tf_dataset(train_d_r, label=train_labels)

tuner = tfdf.tuner.RandomSearch(num_trials=20)

# Hyper-parameters to optimize.
tuner.discret("max_depth", [4, 5, 6, 7])

modelA = tfdf.keras.RandomForestModel(tuner=tuner)
modelA.fit(train_d_r, train_labels)

print(modelA.summary())
```

Use /tmp/tmp383vndj8 as temporary training directory
Reading training dataset...

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
Cell In[165], line 11
```

```
    7 # Hyper-parameters to optimize.
    8 # tuner.discret("max_depth", [4, 5, 6, 7])
   10 modelA = tfdf.keras.RandomForestModel()
--> 11 modelA.fit(train_d_r, train_labels)
   13 print(modelA.summary())
```

```
File ~/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/tensorflow_decision_forests/keras/core.py:1262, in CoreModel.fit(self, x, y, callbacks, verbose, validation_steps, validation_data, sample_weight, steps_per_epoch, class_weight, **kwargs)
```

```
   1259 # Reset the training status.
   1260 self._is_trained.assign(False)
-> 1262 return self._fit_implementation(
   1263     x=x,
   1264     y=y,
   1265     callbacks=callbacks,
   1266     verbose=self._verbose,
   1267     validation_steps=validation_steps,
   1268     validation_data=validation_data,
   1269     sample_weight=sample_weight,
   1270     steps_per_epoch=steps_per_epoch,
   1271     class_weight=class_weight,
   1272 )
```

```
File ~/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/tensorflow_decision_forests/keras/core.py:1403, in CoreModel._fit_implementation(self, x, y, verbose, callbacks, sample_weight, validation_data, validation_steps, steps_per_epoch, class_weight)
```

```
   1396 coordinator = None
   1397 if distribution_config is None or not self.support_distributed_training():
   1398     # Local training.
   1399
   1400     # Wraps the input training dataset into a tf.data.Dataset like object.
   1401     # This is a noop if the training dataset is already provided as a
   1402     # tf.data.Dataset.
-> 1403     data_handler = get_data_handler(
   1404         x=x,
   1405         y=y,
   1406         sample_weight=sample_weight,
   1407         model=self,
   1408         class_weight=class_weight,
   1409     )
   1410     iterator = iter(data_handler._dataset) # pylint: disable=protected-access
   1412     if steps_per_epoch is None:
   1413         # Local training with finite dataset.
   1414
   1415         # Load the entire training dataset in Yggdrasil in a single TensorFlow
   1416         # step.
```

```
File ~/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/keras/src/engine/data_adapter.py:1678, in get_data_handler(*args, **kwargs)
```

```

gs)
1676         return _ClusterCoordinatorExactEvalDataHandler(*args,
**kwargs)
1677     return _ClusterCoordinatorDataHandler(*args, **kwargs)
-> 1678 return DataHandler(*args, **kwargs)

File ~/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/keras/engine/data_adapter.py:1284, in DataHandler.__init__(self, x, y, sample_weight, batch_size, steps_per_epoch, initial_epoch, epochs, shuffle, test_data, max_queue_size, workers, use_multiprocessing, max_steps_per_execution, distribute, pss_evaluation_shards)
1281 else:
test_dir = "/home/ec2-user/SageMaker/test"
-> 1284 adapter_cls = select_data_adapter(x, y)
print(adapter_cls)
1286     x,
demented = "/home/ec2-user/SageMaker/test/demented"
not_demented = "/home/ec2-user/SageMaker/test/not_demented"
1298     pss_evaluation_shards=pss_evaluation_shards,
print(os.listdir(demented))
1301 strategy = tf.distribute.get_strategy()

demented_data = os.listdir(demented)
not_demented_data = os.listdir(not_demented)
-> 1102 adapter_cls = [cls for cls in ALL_ADAPTER_CLS if cls.can_handle(x, y)]
label = if not adapter_cls:
for i in range(len(os.listdir(demented))):
try:
-> 1105 temp_file = demented_data[i]
1106 path = demented + "/" + temp_file
format(pathlib.Path(path))
1107 image_dem = tf.keras.utils.load_img(temp)
1108 data.append(image_dem)
1109 labels.append("demented")
# 1110 elif len(adapter_cls) > 1:
# 1111     raise RuntimeError(labels))
1112     "Data adapters should be mutually exclusive for "
1113     "handling inputs. Found multiple adapters {} to handle "
"    temp_file_nd = not_demented_data[i]
1114 path_temp = not_demented + "/" + temp_file + "_nd_type_name(x), _type_name(y)
tempn = pathlib.Path(path_temp)
1115 image_ndem = tf.keras.utils.load_img(tempn)
data.append(image_ndem)
labels.append("not_demented")
ValueError: Failed to find data adapter that can handle input: <class 'numpy.ndarray'>, (<class 'tuple'> containing values of types {'<class 'str'>'})
print(i)
continue
print(len(data), len(labels))

['not_demented', '.DS_Store', 'demented']
1278
543
583
2553 2553

```

In [167]:

```
previous_img = tf.expand_dims(data[0], 0) # Create a batch

for i in range(1, 2553):
    current_img = tf.expand_dims(data[i], 0) # Create a batch
    previous_img = np.concatenate((current_img, previous_img), axis=0)

# print(previous_img.shape)
f, s, t, fo = previous_img.shape

prev_img = previous_img.reshape(f, (s*t*fo))

print(prev_img.shape)

lab = []
for each in labels:
    if each == "demented":
        lab.append(0)
    elif each == "not_demented":
        lab.append(1)
print(len(lab))
```

```
(2553, 49152)
2553
```

In [168]:

```

predictions = model.predict(prev_img)
# score = tf.nn.softmax(predictions)
# class_index = [np.argmax(x) for x in score]
# class_name = [class_names[index] for index in class_index]

y_pred = predictions
y_test = lab

y_pred = []
for each in predictions:
    if each == "demented":
        y_pred.append(0)
    elif each == "not_demented":
        y_pred.append(1)
print(len(y_pred))

import pandas as pd

print(type(predictions))

pd.Index(list(predictions)).value_counts()

confusion_matrix = tf.math.confusion_matrix(y_pred, y_test)
print(confusion_matrix)

classification_report = classification_report(y_test, y_pred)
print(classification_report)

```

```

2553
<class 'numpy.ndarray'>
tf.Tensor(
[[ 88  82]
 [1189 1194]], shape=(2, 2), dtype=int32)

```

	precision	recall	f1-score	support
0	0.52	0.07	0.12	1277
1	0.50	0.94	0.65	1276
accuracy			0.50	2553
macro avg	0.51	0.50	0.39	2553
weighted avg	0.51	0.50	0.39	2553

CNN model

Train the model for 10 epochs with the Keras `Model.fit` method:

Overfitting generally occurs when there are a small number of training examples. [Data augmentation](#) ([./data_augmentation.ipynb](#)) takes the approach of generating additional training data from your existing examples by augmenting them using random transformations that yield believable-looking images. This helps expose the model to more aspects of the data and generalize better.

You will implement data augmentation using the following Keras preprocessing layers:

`tf.keras.layers.RandomFlip`, `tf.keras.layers.RandomRotation`, and `tf.keras.layers.RandomZoom`. These can be included inside your model like other layers, and run on the GPU.

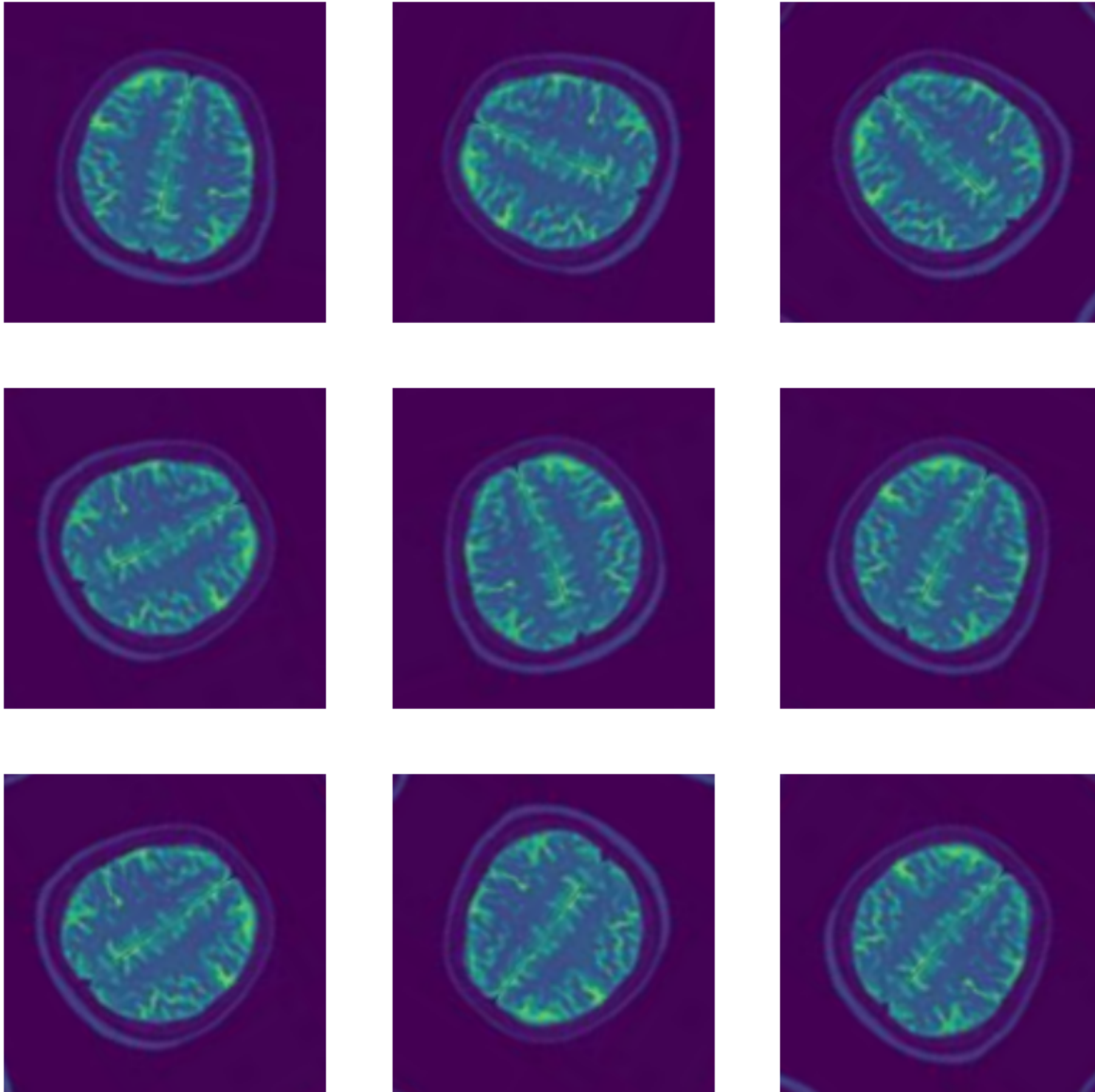
In [169]:

```
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal",
                           input_shape=(img_height,
                                         img_width,
                                         3)),
        layers.RandomRotation(0.7),
        layers.RandomZoom(0.005),
        layers.Resizing(224, 224),
        layers.RandomFlip(mode="horizontal_and_vertical")
    ]
)
```

Visualize a few augmented examples by applying data augmentation to the same image several times:

In [170]:

```
plt.figure(figsize=(10, 10))
for images, _ in train_ds.take(1):
    for i in range(9):
        augmented_images = data_augmentation(images)
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(augmented_images[0].numpy().astype("uint8"))
        plt.axis("off")
```



You will add data augmentation to your model before training in the next step.

Dropout

Another technique to reduce overfitting is to introduce [dropout](https://developers.google.com/machine-learning/glossary#dropout) (https://developers.google.com/machine-learning/glossary#dropout regularizaton) regularizaton to the network.

When you apply dropout to a layer, it randomly drops out (by setting the activation to zero) a number of output units from the layer during the training process. Dropout takes a fractional number as its input value, in the form such as 0.1, 0.2, 0.4, etc. This means dropping out 10%, 20% or 40% of the output units randomly from the applied layer.

Create a new neural network with `tf.keras.layers.Dropout` before training it using the augmented images.

In [178]:

```
num_classes = len(class_names)

CNN = Sequential([
    data_augmentation,
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
    layers.Conv2D(16, 3, padding='same', activation='relu', data_format="channels_last"),
    layers.MaxPooling2D(data_format="channels_last"),
    layers.Conv2D(32, 3, padding='same', activation='relu', data_format="channels_last"),
    layers.MaxPooling2D(data_format="channels_last"),
    layers.Conv2D(64, 3, padding='same', activation='relu', data_format="channels_last"),
    layers.MaxPooling2D(data_format="channels_last"),
    layers.Conv2D(128, 3, padding='same', activation='relu', data_format="channels_last"),
    layers.MaxPooling2D(data_format="channels_last"),
    # layers.Dropout(0.0),
    layers.Flatten(),
    layers.Dense(256, activation='relu', kernel_regularizer='l2'),
    layers.Dense(64, activation='relu', kernel_regularizer='l2'),
    layers.Dense(num_classes, name="outputs")
])

CNN.compile(optimizer='adam',
            loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
            metrics=['accuracy'])

CNN.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 224, 224, 3)	0
rescaling_7 (Rescaling)	(None, 224, 224, 3)	0
conv2d_16 (Conv2D)	(None, 224, 224, 16)	448
max_pooling2d_16 (MaxPooling2D)	(None, 112, 112, 16)	0
conv2d_17 (Conv2D)	(None, 112, 112, 32)	4640
max_pooling2d_17 (MaxPooling2D)	(None, 56, 56, 32)	0
conv2d_18 (Conv2D)	(None, 56, 56, 64)	18496
max_pooling2d_18 (MaxPooling2D)	(None, 28, 28, 64)	0
conv2d_19 (Conv2D)	(None, 28, 28, 128)	73856
max_pooling2d_19 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_4 (Flatten)	(None, 25088)	0
dense_5 (Dense)	(None, 256)	6422784
dense_6 (Dense)	(None, 64)	16448
outputs (Dense)	(None, 2)	130

=====
 Total params: 6536802 (24.94 MB)
 Trainable params: 6536802 (24.94 MB)
 Non-trainable params: 0 (0.00 Byte)

Compile and train the model

In [*]:

```
epochs = 150
history = CNN.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

Epoch 128/150
576/576 [=====] - 8s 13ms/step - loss: 0.5897
- accuracy: 0.6976 - val_loss: 0.5929 - val_accuracy: 0.6965
Epoch 129/150
576/576 [=====] - 8s 13ms/step - loss: 0.5913
- accuracy: 0.7022 - val_loss: 0.5911 - val_accuracy: 0.7000
Epoch 130/150
576/576 [=====] - 8s 13ms/step - loss: 0.5895
- accuracy: 0.6971 - val_loss: 0.6014 - val_accuracy: 0.6941
Epoch 131/150
576/576 [=====] - 8s 13ms/step - loss: 0.5903
- accuracy: 0.7022 - val_loss: 0.5914 - val_accuracy: 0.7009
Epoch 132/150
576/576 [=====] - 8s 13ms/step - loss: 0.5932
- accuracy: 0.6994 - val_loss: 0.6036 - val_accuracy: 0.6963
Epoch 133/150
576/576 [=====] - 8s 13ms/step - loss: 0.5882
- accuracy: 0.7006 - val_loss: 0.6094 - val_accuracy: 0.6859
Epoch 134/150
336/576 [=====>.....] - ETA: 2s - loss: 0.5892 - ac

Visualize training results

After applying data augmentation and `tf.keras.layers.Dropout`, there is less overfitting than before, and training and validation accuracy are closer aligned:

In [181]:

```

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

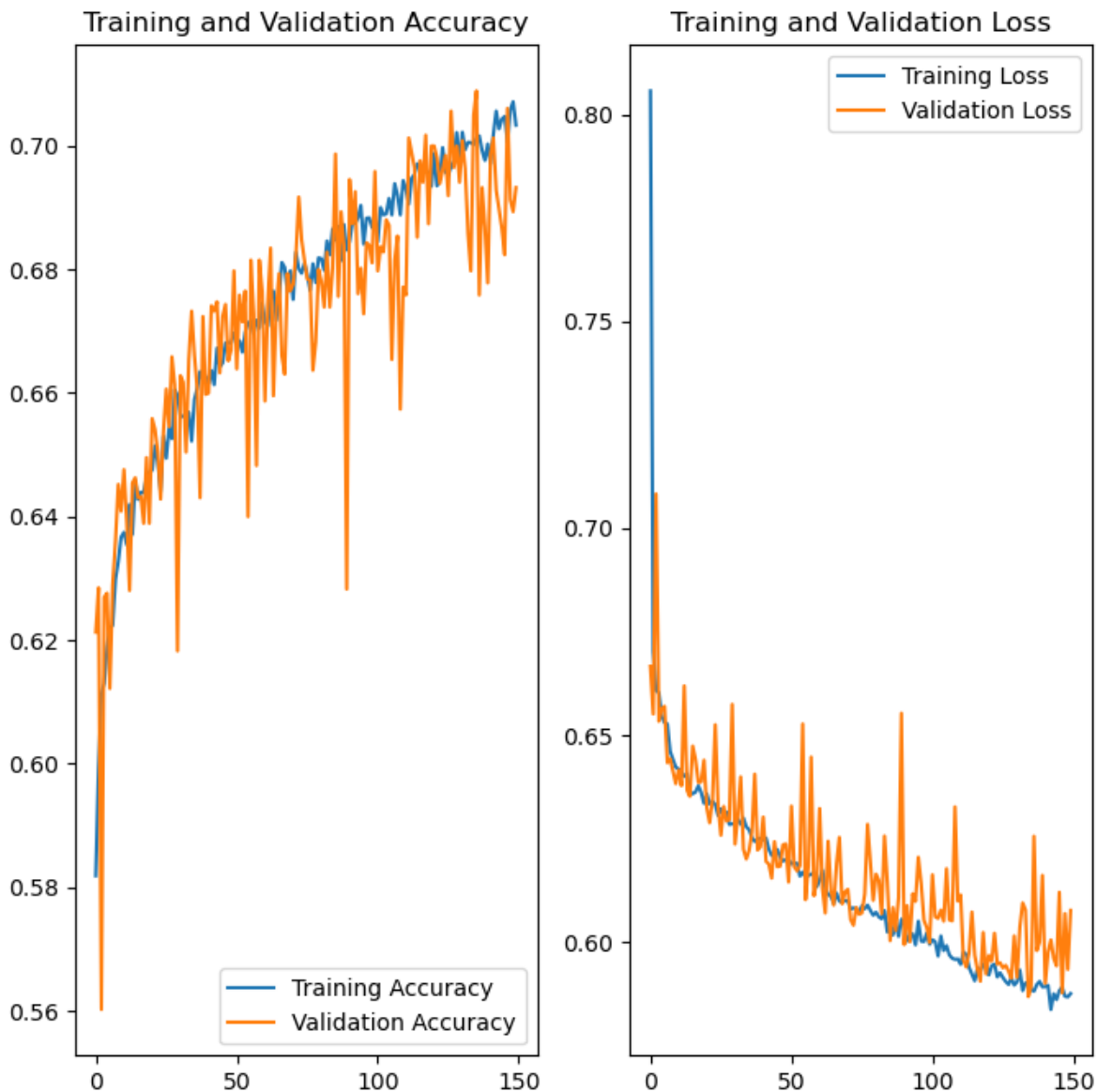
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()

```



Predict on new data

Use your model to classify an image that wasn't included in the training or validation sets.

Note: Data augmentation and dropout layers are inactive at inference time.

In [76]:

```
# import zipfile
# with zipfile.ZipFile("./test.zip", 'r') as zip_ref:
#     zip_ref.extractall("./")
```

In [54]:

```

# Generate Test data
import pathlib

test_dir = "/home/ec2-user/SageMaker/test"

print(os.listdir(test_dir))

demented = "/home/ec2-user/SageMaker/test/demented"
not_demented = "/home/ec2-user/SageMaker/test/not_demented"

print(len(os.listdir(demented)))

demented_data = os.listdir(demented)
not_demented_data = os.listdir(not_demented)

data = []
labels = []
for i in range(len(os.listdir(demented))):
    try:
        temp_file_d = demented_data[i]
        path = demented + "/" + temp_file_d
        temp = pathlib.Path(path)
        image_dem = tf.keras.utils.load_img(temp)
        data.append(image_dem)
        labels.append("demented")
    #     print(image_dem)
    #     print(list(zip(data, labels)))

    temp_file_nd = not_demented_data[i]
    path_temp = not_demented + "/" + temp_file_nd
    tempn = pathlib.Path(path_temp)
    image_ndem = tf.keras.utils.load_img(tempn)
    data.append(image_ndem)
    labels.append("not_demented")
except Exception as err:
    print(i, err)
    continue
print(len(data), len(labels))

```

```

['not_demented', '.DS_Store', 'demented']
1278
543 cannot identify image file <_io.BytesIO object at 0x7fa992a50f90>
583 cannot identify image file <_io.BytesIO object at 0x7fa992ab9df0>
2553 2553

```

In [183]:

```
previous_img = tf.expand_dims(data[0], 0) # Create a batch

for i in range(1, 2553):
    current_img = tf.expand_dims(data[i], 0) # Create a batch
    previous_img = np.concatenate((current_img, previous_img), axis=0)

# print(previous_img.shape)
f, s, t, fo = previous_img.shape
print(np.max(previous_img))
previous_img = previous_img/255
print(np.max(previous_img))

# print(prev_img.shape)

lab = []
for each in labels:
    if each == "demented":
        lab.append(0)
    elif each == "not_demented":
        lab.append(1)
lab = lab
print(len(lab))
```

```
255
1.0
2553
```

In [186]:

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# class_names = ["not_demented", "demented"]

lab = []
for each in labels:
    if each == "demented":
        lab.append(0)
    elif each == "not_demented":
        lab.append(1)
# print(len(lab))

predictions = model.predict(previous_img)
score = tf.nn.softmax(predictions)
class_index = [np.argmax(x) for x in score]

class_name = [class_names[index] for index in class_index]

y_pred = class_index
y_test = lab

confusion_matrix = tf.math.confusion_matrix(predictions = y_test, labels = y_pred)
print(confusion_matrix)

classification_report = classification_report(y_test, y_pred)

print(classification_report)

```

```

80/80 [=====] - 0s 4ms/step
tf.Tensor(
[[ 0  0]
 [1277 1276]], shape=(2, 2), dtype=int32)

```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1277
1	0.50	1.00	0.67	1276
accuracy			0.50	2553
macro avg	0.25	0.50	0.33	2553
weighted avg	0.25	0.50	0.33	2553

```
/home/ec2-user/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
/home/ec2-user/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
    y_pred = y_test[ind], y_pred[ind]
    _warn_prf(average, modifier, msg_start, len(result))
/home/ec2-user/anaconda3/envs/tensorflow2_p310/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
    _warn_prf(average, modifier, msg_start, len(result))
```

Use TensorFlow Lite

TensorFlow Lite is a set of tools that enables on-device machine learning by helping developers run their models on mobile, embedded, and edge devices.

Convert the Keras Sequential model to a TensorFlow Lite model

To use the trained model with on-device applications, first [convert it](https://www.tensorflow.org/lite/models/convert) (<https://www.tensorflow.org/lite/models/convert>) to a smaller and more efficient model format called a [TensorFlow Lite](https://www.tensorflow.org/lite/) (<https://www.tensorflow.org/lite/>) model.

In this example, take the trained Keras Sequential model and use

```
tf.lite.TFLiteConverter.from_keras_model
```

to generate a [TensorFlow Lite](https://www.tensorflow.org/lite/) (<https://www.tensorflow.org/lite/>) model:

In [31]:

```
# Convert the model.
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save the model.
with open('model.tflite', 'wb') as f:
    f.write(tflite_model)
```

WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.

WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.

```

use there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause the
re is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause the
re is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUn
iformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTr
ansformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip ca
use there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause the
re is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause the
re is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUn
iformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTr
ansformV3 cause there is no registered converter for this op.

WARNING:absl:Found untraced functions such as _jit_compiled_convolutio
n_op, _jit_compiled_convolution_op, _jit_compiled_convolution_op while
saving (showing 3 of 3). These functions will not be directly callable
after loading.

INFO:tensorflow:Assets written to: /tmpfs/tmp/tmpr2jwfqh7/assets

INFO:tensorflow:Assets written to: /tmpfs/tmp/tmpr2jwfqh7/assets
2022-12-14 02:30:14.950940: W tensorflow/compiler/mlir/lite/python/tf_
tfl_flatbuffer_helpers.cc:362] Ignored output_format.
2022-12-14 02:30:14.950996: W tensorflow/compiler/mlir/lite/python/tf_
tfl_flatbuffer_helpers.cc:365] Ignored drop_control_dependency.

```

The TensorFlow Lite model you saved in the previous step can contain several function signatures. The Keras model converter API uses the default signature automatically. Learn more about [TensorFlow Lite signatures \(https://www.tensorflow.org/lite/guide/signatures\)](https://www.tensorflow.org/lite/guide/signatures).

Run the TensorFlow Lite model

You can access the TensorFlow Lite saved model signatures in Python via the `tf.lite.Interpreter` class.

Load the model with the `Interpreter` :

In [32]:

```

TF_MODEL_FILE_PATH = 'model.tflite' # The default path to the saved TensorFlow Lite
interpreter = tf.lite.Interpreter(model_path=TF_MODEL_FILE_PATH)

```

Print the signatures from the converted model to obtain the names of the inputs (and outputs):

In [33]:

```
interpreter.get_signature_list()
```

Out[33]:

```
{'serving_default': {'inputs': ['sequential_1_input'], 'outputs': ['outputs']}}
```

In this example, you have one default signature called `serving_default`. In addition, the name of the 'inputs' is 'sequential_1_input', while the 'outputs' are called 'outputs'. You can look up these first and last Keras layer names when running `Model.summary`, as demonstrated earlier in this tutorial.

Now you can test the loaded TensorFlow Model by performing inference on a sample image with `tf.lite.Interpreter.get_signature_runner` by passing the signature name as follows:

In [34]:

```
classify_lite = interpreter.get_signature_runner('serving_default')
classify_lite
```

Out[34]:

```
<tensorflow.lite.python.interpreter.SignatureRunner at 0x7f0364433580>
```

Similar to what you did earlier in the tutorial, you can use the TensorFlow Lite model to classify images that weren't included in the training or validation sets.

You have already tensorized that image and saved it as `img_array`. Now, pass it to the first argument (the name of the 'inputs') of the loaded TensorFlow Lite model (`predictions_lite`), compute softmax activations, and then print the prediction for the class with the highest computed probability.

In [35]:

```
predictions_lite = classify_lite(sequential_1_input=img_array)['outputs']
score_lite = tf.nn.softmax(predictions_lite)
```

In [36]:

```
print(
    "This image most likely belongs to {} with a {:.2f} percent confidence."
    .format(class_names[np.argmax(score_lite)], 100 * np.max(score_lite))
)
```

```
This image most likely belongs to sunflowers with a 97.33 percent confidence.
```

The prediction generated by the lite model should be almost identical to the predictions generated by the original model:

In [37]:

```
print(np.max(np.abs(predictions - predictions_lite)))
```

```
1.4305115e-06
```

Of the five classes— 'daisy' , 'dandelion' , 'roses' , 'sunflowers' , and 'tulips' —the model should predict the image belongs to sunflowers, which is the same result as before the TensorFlow Lite conversion.

Next steps

This tutorial showed how to train a model for image classification, test it, convert it to the TensorFlow Lite format for on-device applications (such as an image classification app), and perform inference with the TensorFlow Lite model with the Python API.

You can learn more about TensorFlow Lite through [tutorials \(https://www.tensorflow.org/lite/tutorials\)](https://www.tensorflow.org/lite/tutorials) and [guides \(https://www.tensorflow.org/lite/guide\)](https://www.tensorflow.org/lite/guide).